WEEK 6

Large-Scale Data Pipelines and Orchestration

Large-Scale Data Computing

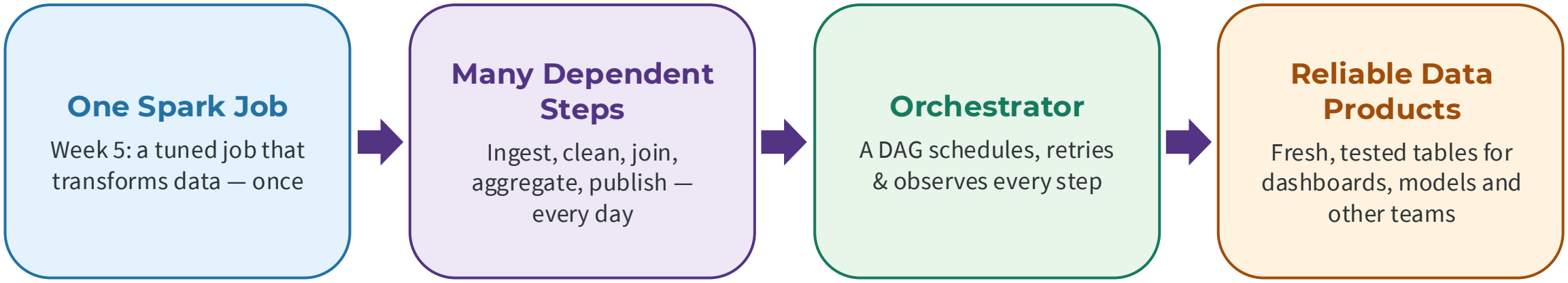
Orchestrated Pipelines
ETL/ELT patterns composed as DAGs

Reliable by Design
Idempotency, retries & backfills

Trusted Data
Quality gates & tested pipelines

CDLE • MEIC/MEIM • ISEL

From Jobs to Pipelines



This week: from writing jobs to running platforms
A **pipeline** is code plus **time**, **failure** and **change** — **orchestration** is how we engineer for all three.

NEW TERMS

- `Pipeline` chain of dependent jobs run on a schedule
- `Orchestrator` schedules, retries and tracks every step
- `DAG` directed acyclic graph of tasks
- `Data product` a tested table consumers rely on

Contents

Everything for week 6 — pipeline patterns, orchestration, reliability, and data you can trust

01 ETL / ELT Patterns

- Extract–transform–load vs load-then-transform
- Staging layers: bronze → silver → gold
- Anatomy of a modern pipeline
- Where the week-4/5 Spark jobs fit in

02 Orchestration & DAGs

- Why cron is not enough
- Airflow-style DAGs: tasks & dependencies
- Inside an orchestrator: scheduler & task states
- Operators, sensors & scheduling

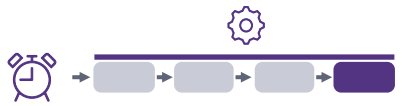
03 Reliability Engineering

- Idempotency — the golden rule
- Retries that are safe to run
- Backfills & reprocessing history
- Observability of pipeline runs

04 Data Quality & Lab

- Quality gates & expectation frameworks
- Pipeline testing & AI-assisted rules
- The orchestration ecosystem
- Lab: an orchestrated multi-stage pipeline

Pipelines in Practice



Four common pipelines: different data, the same shape of sources → steps → output on a schedule

Nightly sales daily at 02:00	store + web orders	clean & dedupe	aggregate by day	**BI dashboard**
ML features every hour	clickstream events	sessionise	compute features	**model scoring**
Reconciliation daily, hard deadline	bank feeds + ledger	match entries	flag mismatches	**exceptions report**
IoT sensors every 15 minutes	device readings	deduplicate	hourly aggregates	**alerts + archive**

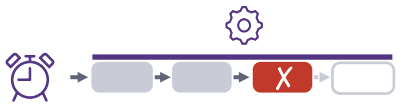
All four share **a schedule, dependent steps, failures mid-run and data someone must trust**.

Today: building pipelines that are **timely**, **modular**, **idempotent** and **correct**, through orchestration, DAGs, retries & backfills, quality gates.

NEW TERMS

- **`Warehouse`** central store of analytics tables
- **`Dependency`** a step that must finish first
- **`Dedupe`** remove duplicate records (deduplicate)
- **`BI dashboard`** charts business users read daily
- **`Idempotent`** safe to rerun: same result every time
- **`Sessionise`** group a user's events into visits/sessions
- **`Reconciliation`** match bank records to the ledger
- **`Ledger`** company's own record of every transaction

Scenario — The 3 a.m. Failure



A nightly pipeline dies halfway through — what happens to yesterday's data?

The Incident

"The 02:00 sales pipeline failed at step 3 of 5 — the dashboard is wrong and the board meets at 9."

- Steps 1–2 finished and already wrote rows into the warehouse
- A blind re-run would **`double-count`** rows from steps 1–2
- Fixing it by hand at 3 a.m. is how outages become habits
- The recovery must be automatic, safe and repeatable

Ingest	Clean	Join	Aggregate	Publish
✓ done	✓ done	✗ failed	waiting	waiting

The Real Questions

```
# what on-call needs to know:
#   which step failed, for which day?
#   can step 3 re-run without harm?
#   are steps 1-2 results still valid?
safe recovery — requires:
declared dependencies between steps,
  idempotent steps that re-run safely,
  retries & backfills built in,
  and quality gates = trust
→ that is what orchestration does
```

The lesson

At scale, failure is routine — pipelines must be designed so any step can fail, retry and re-run without corrupting data.

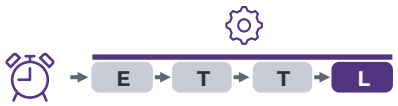
Pipeline vs orchestration

`Pipeline:` the steps that transform the data
`Orchestration:` runs them in order, retries, alerts

NEW TERMS

`Warehouse` central store of analytics tables
`On-call` engineer paged when production breaks
`Idempotent` safe to run twice, same result
`Retry` run a failed step again automatically
`Backfill` re-run a pipeline for past dates
`Quality gate` check that blocks bad data downstream

ETL — The Three Verbs



Every pipeline gets data out, reshapes it and puts it where people read it: the same steps as the four pipelines

Extract get data out of sources	**Sources belong to others.** They change without notice, and your reads must not overload them.	store + web orders · bank feeds · device readings
Transform clean, join, aggregate	**Business logic lives here, and so do most bugs.** The failed Join in the 3 a.m. incident was a transform.	clean & dedupe · match entries · compute features
Load write where people read	**Users see the load.** A half-finished or doubled load is what lands on the dashboard.	BI dashboard · alerts · exceptions report

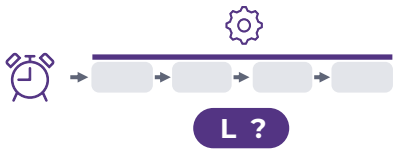
Each verb is a different place to fail: **sources change, logic has bugs, loads double-count**.

The verbs never change. The design choices: where Load writes, and whether Transform comes before or after it.

NEW TERMS

- `Source system` app or database the data comes from
- `Conform` make sources agree on IDs, units, formats
- `Target` where loaded data lives (warehouse, lake)
- `Business logic` rules that turn raw data into answers

Three Places Load Can Write



Each target fixed the gap the one before it left

1990s onward

Data warehouse

Analytics database: data is cleaned and modelled before it lands, and one engine stores and queries it.

Solved: trusted, fast SQL reporting

Gap: costly; holds only data modelled up front

~2010: Hadoop, then S3

Data lake

Cheap file store that keeps data raw, in any format; structure is applied only when it is read.

Solved: all history, any format, for ML

Gap: no transactions or schema: a data swamp

~2020: Delta Lake, Iceberg

Lakehouse

Lake files plus a table log that adds transactions and schema, so any engine reads them as tables.

Solved: warehouse trust at lake cost

Gap: table upkeep, e.g. compacting small files

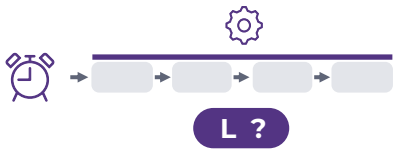
The warehouse gave trust, the lake gave cheap raw storage; **the lakehouse gives both**.

The three side by side: next.

NEW TERMS

- `Object storage` cloud file store (S3, GCS)
- `Schema-on-read` structure applied when data is read
- `Table format` log turning files into tables (Delta, Iceberg)
- `Data swamp` lake data nobody can find or trust

Warehouse vs Lakehouse



Where Load writes decides what you can store, trust and replay

	Data warehouse	Data lake	Lakehouse
Stores	curated tables	raw files of any kind	open files + a table log
Examples	Snowflake, BigQuery, Redshift	Amazon S3, ADLS, GCS	Databricks, Delta Lake, Iceberg
Guarantees	transactions, enforced schema	none: risk of a data swamp	transactions on lake files
Cost & scale	higher; storage tied to the engine	cheapest; scales without limit	lake cost, any engine
Best for	BI and SQL reporting	ML and raw history	BI and ML from one copy
Pipeline fit	ETL	ELT, but fragile	**ELT, safely**

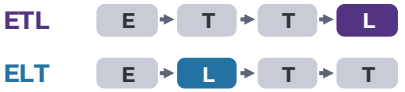
Cheap storage made keeping raw data affordable; **the table log made it trustworthy**.

Together they made ELT the default on the lakehouse: ETL vs ELT, next.

NEW TERMS

`ACID` each write fully succeeds or fully fails

`Enforced schema` rejects writes that break the table



ETL vs ELT — Pipeline Patterns

The three verbs in two orders — the question is where transformation happens

ETL — transform first

```
# clean before it lands
extract(sources)
  .transform(clean, conform)
  .load(warehouse)
# only curated data stored
```

ELT — load first

```
# land raw, refine in place
extract(sources)
  .load(lakehouse, raw)
  .transform with Spark / SQL
# raw history kept — replayable
```

The practical difference

```
transform = before vs after
storage   = curated vs raw
replay    = hard vs anytime
home      = DW vs lakehouse
```

✓ **ELT rules the lakehouse**

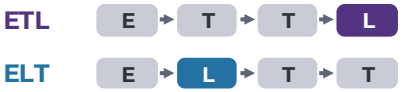
✓ **Raw history = replay**

✓ **Same verbs, new order**

Cheap object storage tilted the industry towards **ELT** — keep the raw history, transform in place, re-derive anything.
ETL still wins when data must be cleaned before it may be stored — privacy, contracts, compliance.

NEW TERMS

- **ETL** Extract, Transform, then Load
- **ELT** Extract, Load, then Transform in place
- **Curated** cleaned, approved, ready for use
- **ETLT** light clean before load, rest after

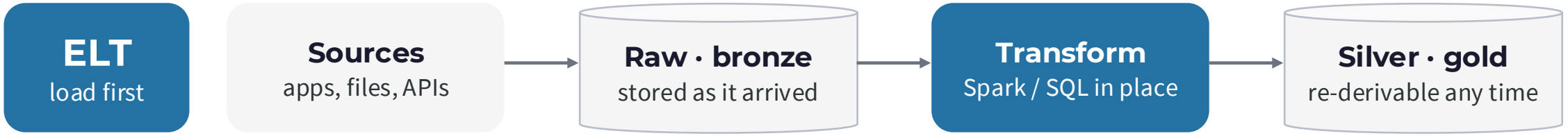


ETL vs ELT — The Data Flow

Same verbs, different order — what lands in storage decides what you can replay



A logic bug → **re-extract from the sources**, if they still hold the data



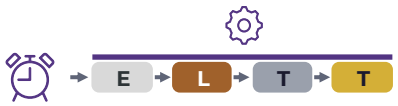
A logic bug → **re-run the transform over bronze**; the raw copy never left

Load before you transform and the raw history becomes a replay source. That is why lakehouse pipelines default to **ELT**.

NEW TERMS

- `Bronze` raw layer, stored as it arrived
- `Silver` cleaned, deduplicated, conformed data
- `Gold` business-ready tables for reports and models
- `Replay` re-run transforms over stored raw data

Anatomy of a Modern Pipeline



Four stages, one direction — each layer adds structure, trust and value

1 → E + L → 2 → T → 3 → T → 4

Ingest · raw / bronze
Land source data exactly as it arrived — immutable, append-only, replayable

`bronze.sales_raw`

Clean · silver
Validate, deduplicate and conform types & schemas into consistent tables

`silver.sales`

Transform · gold
Join and aggregate into business-level tables, ready for consumption

`gold.daily_sales`

Serve & observe
Publish to BI, ML and APIs — with lineage, run metrics and quality checks

`dashboard · ML · API`

Why layers matter
When stage 3 (gold) fails, bronze and silver still hold good data, so recovery restarts from the last trusted layer.
This “medallion” pattern (**bronze** (raw) → **silver** (cleaned) → **gold** (transformed)) is the default shape of ELT pipelines on the lakehouse.

NEW TERMS

- `Medallion` the bronze → silver → gold layering
- `Immutable` written once, never changed
- `Conform` align types and schemas across sources
- `Lineage` record of which data produced which

Orchestration — Key Concepts

```
# “cron + scripts” looks fine…
0 2 * * *  run_pipeline.sh
# …until you ask:
#  did yesterday's run finish?
#  step 3 failed — retry only step 3?
#  who re-runs March after a fix?
orchestrator = execution with
  memory, dependencies &
  recovery — cron has none
# then recovery is one command:
$ airflow backfill create
  --from-date 2026-03-01 --to-date 2026-03-31
  --dag-id daily_sales
```

The Vocabulary

DAG → tasks + dependencies, no cycles

Task → one unit of work — one step

Schedule → when each run should start

Run → one dated execution of the DAG

Sensor → wait for data before starting

Backfill → re-run a range of past dates

Idempotent → safe to run twice — slide 24

SLA → the moment “late” becomes an incident

A scheduler answers “when”. An orchestrator also answers “in what order, what failed, and how do we recover” — that difference is this whole lecture.



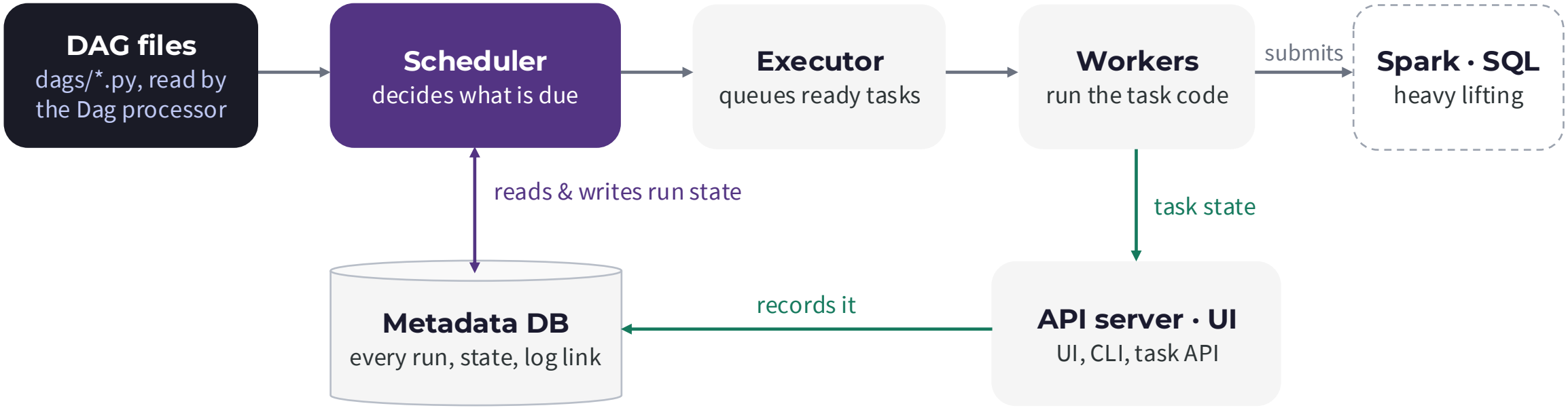
NEW TERMS

cron Unix scheduler: run a command at set times
0 2 * * * cron expression: daily at 02:00
m h d m wd min hour day month weekday

Airflow open-source Python orchestrator
SLA Service Level Agreement: agreed deadline for data
Deadline Alert Airflow 3.1+ successor to SLA alerts

Inside an Orchestrator

The scheduler decides, workers execute, and the API server records each result in the metadata DB



The scheduler never runs your code. It **records state** and hands ready **tasks to workers**, and **that record is what makes retries and backfills possible**.

NEW TERMS

- `Scheduler` decides which tasks are due
- `Executor` hands ready tasks to workers
- `Worker` process that runs the task code
- `Metadata DB` database holding every run's state
- `API server` door to the metadata DB for workers and UI

Apache Airflow 3

What Airflow is, and why “workflows as code” is its core idea

Apache Airflow® is an open-source **platform** for **developing**, **scheduling**, and **monitoring workflows**, such as traditional time based or event-triggered batch-oriented data pipelines, machine learning, model training, and agentic or LLM-based workloads.

Airflow’s extensible Python framework enables you to **build workflows connecting with virtually any technology**, with a growing set of providers for orchestrating AI and agentic tools alongside the rest of your pipeline.

A web-based UI helps you visualize, manage, and debug your workflows. You can run Airflow in a variety of configurations — from a single process on your laptop to a distributed system capable of handling massive workloads.

Airflow workflows are defined entirely in Python. The “workflows as code” approach brings these advantages:

Dynamic

Pipelines are defined in code, enabling dynamic Dag generation and parameterization.

Extensible

The Airflow framework includes a wide range of built-in operators and can be extended to fit your needs.

Flexible

Airflow leverages the **Jinja** templating engine, allowing rich customizations.

Source: Apache Airflow documentation, “What is Airflow?” (Airflow 3.x)

NEW TERMS

- `Workflows as code` pipelines defined in versioned files
- `Provider` package of operators for one technology
- `Dynamic Dag` Dag generated by code, e.g. in a loop
- `Jinja` templating engine used in task fields

Airflow 3 Components

The orchestrator behind Lab 6: the parts from slide 13, with their Airflow names

What it is

- Open source, started at Airbnb in 2014
- Apache top-level project since 2019
- A Dag is a Python file in `dags/`
- **Airflow 3 (2025):** new UI, Dag versioning, assets, a separate Dag processor

Slide 13 part	Airflow 3 component	Started with
Scheduler	**scheduler**	`airflow scheduler`
(parses Dag files)	**Dag processor**	`airflow dag-processor`
Executor	**Local · Celery · Kubernetes**	`config: [core] executor`
Workers	**Celery workers or K8s pods**	`via the Task SDK`
Metadata DB	**PostgreSQL (usually)**	`airflow db migrate`
API server · UI	**API server + React UI**	`airflow api-server`
Sensor waits	**triggerer**	`airflow triggerer`

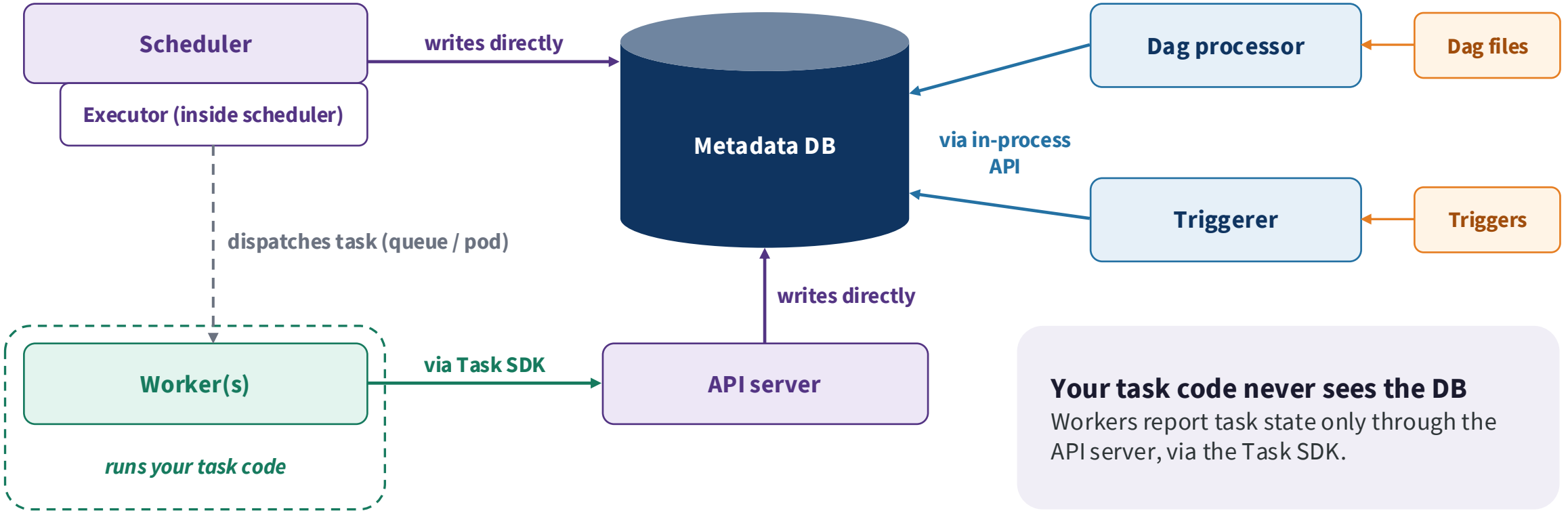
```
# Debug one task for one date: runs the code, records nothing
$ airflow tasks test daily_sales clean 2026-03-01
# Rerun the whole Dag for 5 past dates: one recorded run per date
$ airflow backfill create --dag-id daily_sales --from-date 2026-03-01 --to-date 2026-03-05
```

NEW TERMS

`Dag processor` parses Dag files for the scheduler
`Triggerer` runs deferred waits such as sensors
`Celery` distributed task queue; workers pull tasks
`API server` serves the UI and REST API
`Asset` data item whose update can start a run
`K8s pod` container Kubernetes starts for one task

Airflow 3 Architecture

Only Airflow's own services touch the metadata database



Your task code never sees the DB
Workers report task state only through the API server, via the Task SDK.

Adapted from: Apache Airflow documentation, Architecture Overview (Airflow 3.x)

NEW TERMS

- `Task SDK` library tasks use to talk to Airflow
- `Trigger` async check a deferred task waits on
- `In-process API` API built into the component itself
- `Deferred task` task paused until its trigger fires

Your First Airflow 3 Dag

One Dag, two tasks, two ways to define a task: an operator and a TaskFlow function

dags/demo.py

```python
from datetime import datetime
from airflow.sdk import DAG, task

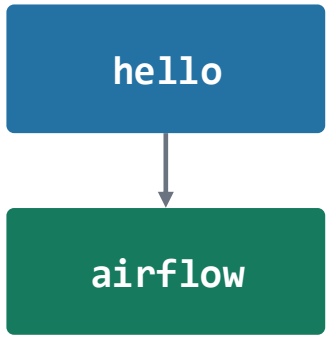
# A Dag represents a workflow, a collection of tasks
with DAG(dag_id="demo", start_date=datetime(2027, 1, 1),
                        schedule="0 0 * * *") as dag:

    # Task 1 - Tasks can be represented as operators
    hello = BashOperator(task_id="hello", bash_command="echo hello")

    # Task 2 - Tasks can be also defined with decorators
    @task()
    def airflow():
        print("airflow")

    # Set dependencies between tasks - the DAG
    hello >> airflow()
```

The graph Airflow builds



defined with **BashOperator**

defined with **@task**

Try it once, locally

$ airflow dags test demo

one run, no scheduler needed. Next slide: the same run in the UI.

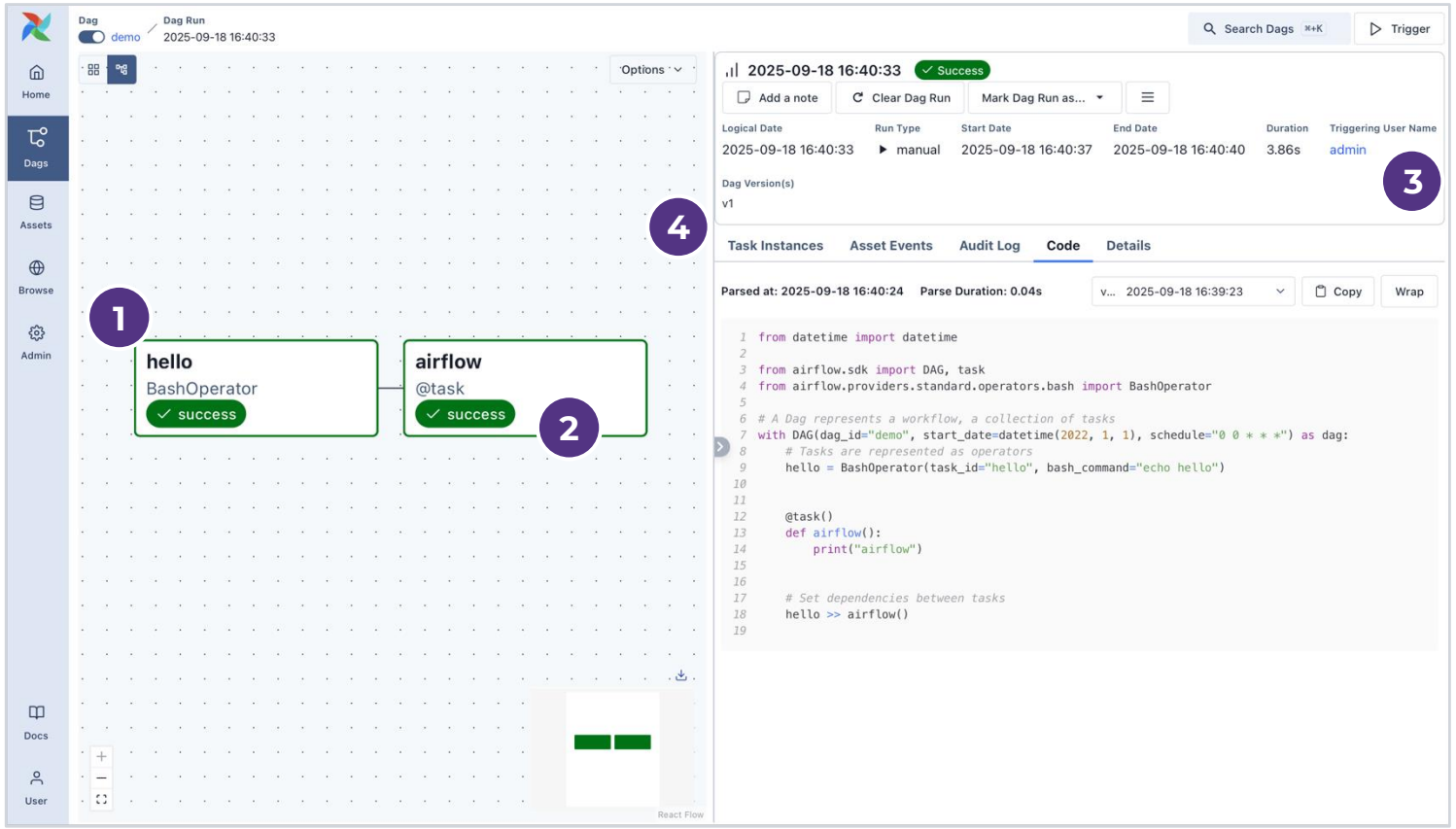
NEW TERMS

- **BashOperator** runs a shell command
- **TaskFlow** tasks as Python functions
- **@task** makes a function a task
- **dags test** one run, no scheduler
- **>>** left task runs first, sequencial
- **start_date** first logical date

- 0 9 * * 1-5 → às 9:00, de segunda a sexta
- */15 * * * * → de 15 em 15 minutos
- 0 0 1 * * → meia-noite do dia 1 de cada mês

The Same Dag in the Airflow 3 UI

One manual run of demo: the graph on the left, the run's details and code on the right



Screenshot: Airflow 3 web UI, Dag run of demo (Graph view and Code tab)

1 Graph view
each box is a task, labelled with how it was defined

2 Task status
green success: the states from the lifecycle slide

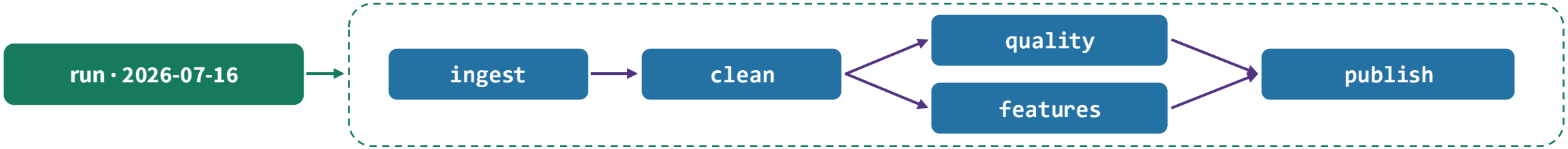
3 Run details
logical date, run type manual, duration 3.86 s

4 Dag version
v1: Airflow 3 keeps every version of the code

NEW TERMS

`Graph view` UI page drawing a Dag's tasks
`Dag run` one execution of a Dag
`Run type` manual, scheduled or backfill
`Dag version` saved copy of the Dag code per change

Airflow 3 DAGs



Run = one dated execution | Task = node | Dependency = edge | Fan-out, then fan-in

The pipeline, as code

```python
# dags/daily_sales.py
from airflow.sdk import DAG
with DAG("daily_sales", start_date=datetime(2026, 1, 1),
         schedule="0 2 * * *") as dag:
    ingest   = SparkSubmitOperator(…)
    clean    = SparkSubmitOperator(…)
    features = SparkSubmitOperator(…)
    quality  = ValidateOperator(…)  # custom
    publish  = DeltaWriteOperator(…)  # custom
    ingest >> clean >> [quality, features] >> publish
    # quality ‖ features in parallel, then publish
```

"Workflows as code"
The DAG file is the single source of truth for what runs, when, and in what order.

What the DAG buys you

- The orchestrator sees what can run in parallel
- A failed task retries alone — not the whole pipeline
- Every run is recorded: inputs, logs, duration, state
- Backfills become a command, not a lost weekend
- Pipelines are code — reviewed, versioned & tested

DAG rule: one level per line

✗ `a >> [b >> c, d] >> e`
✓ `a >> [b, d]; b >> c; [c, d] >> e`

NEW TERMS

`Operator` template for one kind of task
`SparkSubmitOperator` submits a Spark job
`Fan-out / fan-in` one task feeds many / many feed one
`Trigger rule` when a task may run; default all_success

Schedules, Sensors & Operators

SCHEDULES	LOGICAL DATE	POOLS & LIMITS	SENSORS
Time or events Cron expressions, intervals or asset updates launch runs	**Runs are dated** Each run processes its own date's data — the key to backfills	**Protect resources** Limit concurrency so clusters and databases survive load spikes	**Wait for data** Start only when the input partition or file actually exists

OPERATORS	RETRIES	ALERTS & DEADLINES	METADATA
Do one thing Run a Spark job, a SQL script, a container — one task, one operator	**Policy, not code** retries=3 with backoff — declared once, enforced on every task	**Late = incident** Missed deadlines page a human before the consumers notice	**Runs remembered** State, logs and small results are stored for every task run

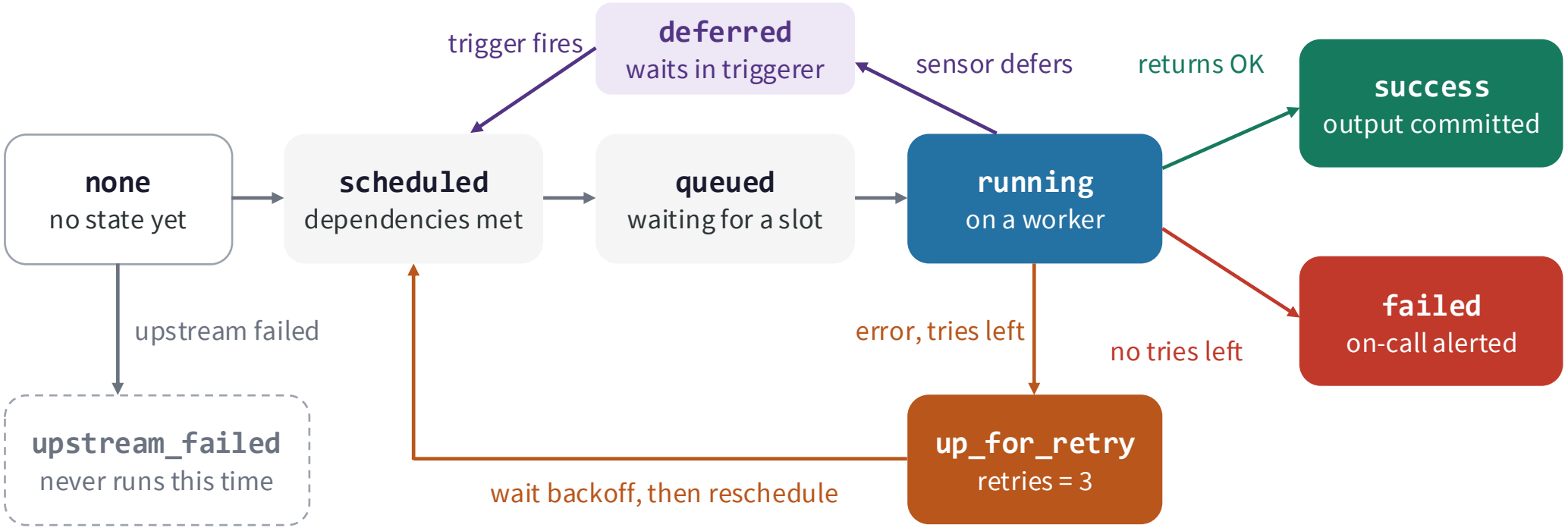
Declarative beats imperative — describe retries, waits and limits once, and the orchestrator enforces them on every single run.

NEW TERMS

- `Logical date` the date a run processes, not when it runs
- `Backoff` growing wait between retries
- `Pool` cap on tasks sharing one resource
- `Asset event` an asset update that starts a run

Task Lifecycle — States of a Run

Every task instance moves through the same states, and the orchestrator records each move



Each state change lands in the metadata DB, so a rerun **repeats only the tasks that did not succeed**.

NEW TERMS

- `Task instance` one task within one DAG run
- `up_for_retry` failed, waiting to try again
- `backoff` growing wait between retries (5 → 10 → 20 min)
- `upstream_failed` not run: a **task before it** failed
- `deferred` paused in the triggerer; frees its worker slot

Airflow 3 — How It Really Runs Your Code

Your file is re-parsed

The Dag processor re-imports every file in **`dags/`** about every 30 s. **Top-level code runs on every parse**, not once per run. Top level code should describe what the DAG is; tasks should do the work.

```
## bad — runs at every parse
rows = db.query(...)

## good — runs once per run
@task
def load():
    rows = db.query(...)
```

Tasks share nothing

Each task is its own process, maybe on another worker. **Return values travel as XComs** through the metadata DB.

```
## bad — data through XCom
return df   # 2 GB

## good — pass a pointer
df.write(path)
return path
-- data lives in storage
```

A task can run twice

A worker that dies misses its heartbeat, and the scheduler re-queues the task. **Delivery is at-least-once**, not exactly-once.

```
## bad — blind append
INSERT INTO gold.daily …

## good — idempotent write
INSERT OVERWRITE …
  PARTITION (day='{{ ds }}')
-- details: slide 24
```

Write every task as if **its file is read constantly, it runs anywhere, and it may run twice**.

NEW TERMS

- **`XCom`** small value passed between tasks via the DB
- **`Heartbeat`** worker's periodic “still alive” signal
- **`At-least-once`** runs one or more times, never zero
- **`Top-level code`** code outside any task; runs at parse

One Dag, All Three Rules (XCom)

```python
from datetime import datetime
import pandas as pd
from airflow.sdk import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
SQL = "SELECT * FROM orders WHERE day = %(d)s"  # 1 cheap
@dag(schedule="@daily", start_date=datetime(2026, 3, 1))
def daily_sales():
    @task
    def extract(ds=None) -> str:   # ds = logical date
        hook = PostgresHook("shop_db")    # 1 connect at run time
        df = hook.get_pandas_df(SQL, parameters={"d": ds})
        # 3 one file per date: a rerun overwrites it
        path = f"s3://lake/bronze/orders/{ds}.parquet"
        df.to_parquet(path)   # 2 the GBs go to storage
        return path           # 2 only the path goes via XCom
    @task
    def clean(path: str) -> str:  # gets a path, not data
        df = pd.read_parquet(path)  # 2 reads the file itself
        out = path.replace("bronze", "silver")
        df.drop_duplicates("order_id").to_parquet(out)
        return out
    clean(extract())   # extract → clean, path via XCom
daily_sales()   # registers the Dag
```

1. **Keep top level cheap**
   Only imports and the SQL string run at parse time. The connection opens inside extract, when the task runs.
2. **Pass a pointer, not data**
   The DataFrame goes to S3; XCom carries a 60-byte path. clean reads the file, maybe on another worker.
3. **Key every output by date**
   ds is in the path: a retry or backfill overwrites the file, never adds a copy.

`A >> B` order only
`B(A())` order + return value via XCom

```python
s = clean(ingest())
publish(features(s), quality(s))
```

→ fan-out from s, fan-in at publish (slide 19)

NEW TERMS

- **`TaskFlow`** @dag and @task turn functions into a Dag
- **`Hook`** ready-made client for an external system
- **`ds`** the run's logical date, as YYYY-MM-DD
- **`s3fs`** lets pandas read and write s3:// paths

Idempotency — The Golden Rule

same input → same state, every run

```
## bad — blind append
INSERT INTO gold.daily
SELECT … FROM silver.events
WHERE day='{{ ds }}'
-- re-run ⇒ day's rows duplicated
## good — own your partition
INSERT OVERWRITE TABLE gold.daily
  PARTITION (day='{{ ds }}')
SELECT … FROM silver.events
WHERE day='{{ ds }}'
-- re-run ⇒ identical state
-- {{ ds }} = the run's logical date
# also: MERGE on keys · write-audit-publish
```

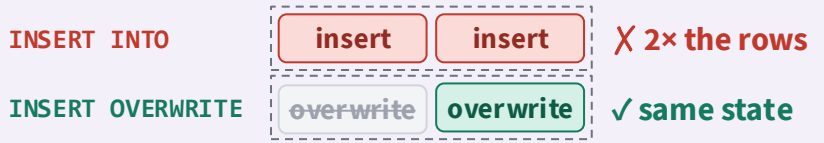
What to notice

Overwrite, don't append
Each run owns its date's partition — running it twice is harmless

Deterministic inputs
Read the run's logical date — never “whatever has arrived so far”

Retries become free
When every task is idempotent, the orchestrator can retry fearlessly

Same day, run twice, one partition



Idempotency is what makes retries and backfills safe — design it into every task from the first line, not after the first outage.
Partition = one folder per day (`gold/daily/day=2026-07-16/`): a run reads and replaces **only its own folder**.

NEW TERMS

- **`Partition`** slice of a table, e.g. one day
- **`INSERT OVERWRITE`** replace a partition's rows
- **`MERGE`** upsert: update or insert on keys
- **`Write-audit-publish`** stage, check, then expose data

Retries & Backfills

```
task clean_events failed — run 2026-07-16
+ policy: retries=3, backoff 5 → 10 → 20 min
+ history: March shipped with a logic bug
```

✓ What a retry fixes

transient faults — lost executors, timeouts, flaky networks: most 3 a.m. failures

same run, same date — the task re-runs its logical date; idempotent writes keep it safe

no human involved — backoff plus alert-on-final-failure lets on-call sleep

✗ When you need a backfill

bad logic, "green" runs — the code was wrong for weeks; history must be recomputed

one command, many runs — the orchestrator replays March day by day, capacity-limited

bounded blast radius — layered tables mean recomputing from silver, not from raw

Retries heal runs; backfills heal history. **Both are only safe because every task is idempotent.**

NEW TERMS

Transient fault temporary error a retry can fix
Blast radius how much data a failure affects
Green run succeeded, but not necessarily correct

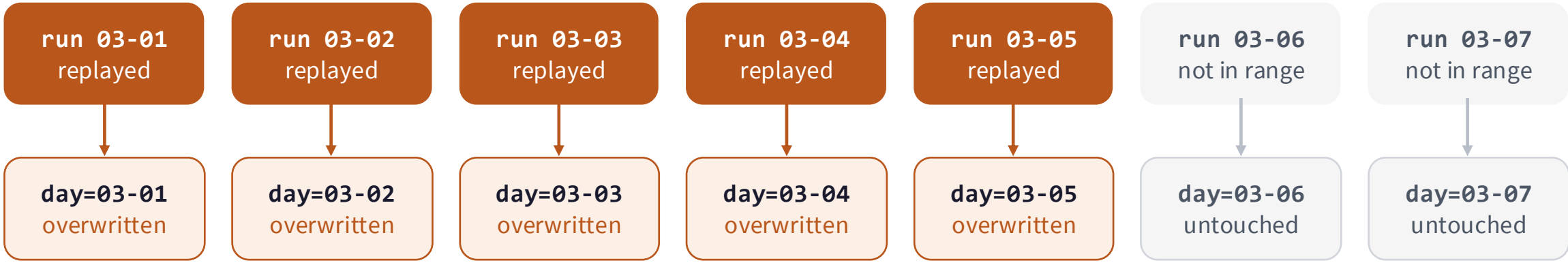
Retry automatic re-run of a **failed** task in the **same run**
Backfill deliberate re-run of **past dates**, even green ones

Backfill — One Run per Logical Date

A backfill is many ordinary runs with past dates; each one rewrites only its own partition

```
$ airflow backfill create --dag-id daily_sales --from-date 2026-03-01 --to-date 2026-03-05
```

DAG runs one per logical date, up to max_active_runs at once



gold.daily partitions `INSERT OVERWRITE … PARTITION (day = {{ ds }})`

Five runs, five partitions, nothing else touched. **Idempotent tasks are what make replaying history safe**.

NEW TERMS `max_active_runs` cap on DAG runs at the same time

Data Quality Gates

THE GATE

Validate between stages

data must pass its checks before it may flow downstream

- **Block, don't warn** a failed check stops the pipeline before bad data reaches gold
- **Gate every boundary** bronze → silver → gold: each promotion must earn its trust
- **Quarantine failures** route bad rows aside for inspection — never silently drop them

EXPECTATIONS

Quality as code

declarative, versioned rules an engine checks on every run

- **Declare** expect order_id unique, never null; revenue ≥ 0 — readable by humans and engines
- **Execute** expectation frameworks (Great Expectations, Soda) run the suite as a task in the DAG
- **Document** every run produces a data-docs report — evidence of quality, not vibes

Golden rule: no data ships until its expectations pass.

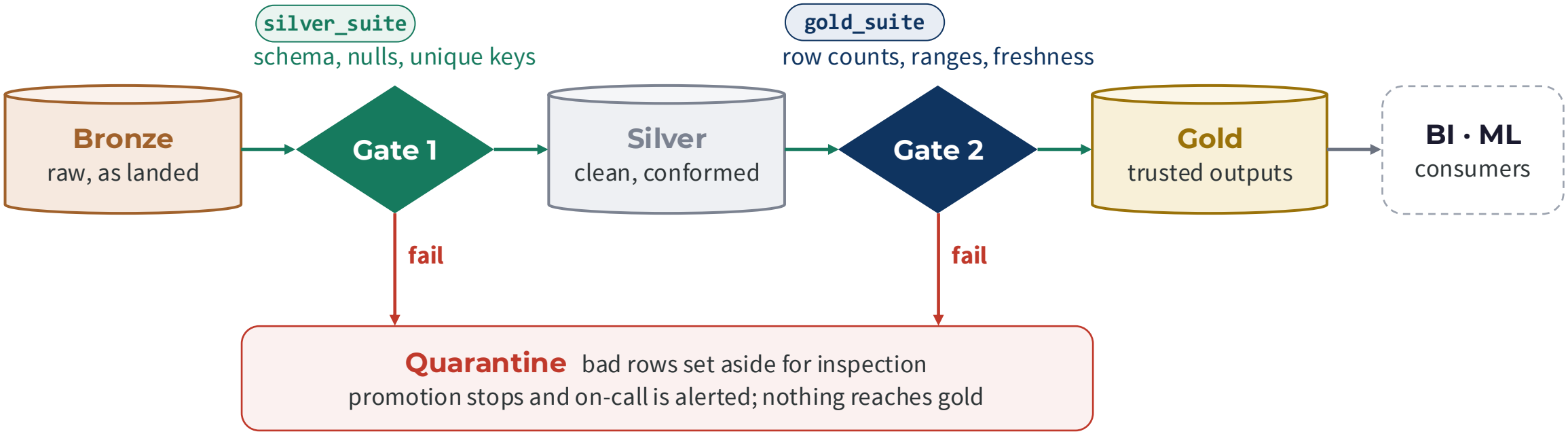
A pipeline without quality gates is just a very fast way to publish wrong numbers.

NEW TERMS

- `Expectation` declarative rule data must satisfy (any tool)
- `Quarantine` set bad rows aside for inspection
- `Great Expectations` Python data-validation framework
- `Data docs` report generated by each validation run

Quality Gates in the Flow

An expectation suite runs at every layer boundary; data moves on only when it passes



A failed check blocks the next layer, so **gold only ever holds data that passed every gate**.

NEW TERMS

- `Freshness` how recently the data was updated
- `Promotion` moving data to the next layer
- `BI` Business Intelligence: dashboards and reports
- `ML` Machine Learning: models trained on the data

Quality Gates — What to Check

Gate 1 checks the shape of the data; Gate 2 checks whether the result is plausible

Gate 1 · bronze → silver: the shape

Schema columns and types as declared
`amount is DECIMAL, not "12,50"`

Nulls required fields always present
`store_id IS NOT NULL`

Unique keys no duplicate source records
`one row per order_id`

```
checks for silver_orders:      # silver_suite
  - missing_count(store_id) = 0
  - duplicate_count(order_id) = 0
```

Gate 2 · silver → gold: is it plausible?

Row counts volume close to normal
`orders within ±30% of same weekday`

Ranges values make business sense
`revenue_eur >= 0; basket €1–2,000`

Freshness data on time
`partition exists before 07:00 SLA`

Grain one row per store and day
`catches a blind INSERT INTO`

```
checks for gold_daily:         # gold_suite
  - min(revenue_eur) >= 0
  - freshness(loaded_at) < 7h
```

The dangerous failures are **green**: the run succeeds, but the numbers are wrong.
Two thresholds: **warn** alerts and still publishes; **fail** quarantines and stops.

NEW TERMS

- **Schema drift** source changes columns or types
- **Grain** what one row means, e.g. store × day
- **Referential integrity** every key exists in its lookup table
- **Threshold** limit that triggers warn or fail

Testing Pipelines Like Software

Four layers of confidence — from a unit test on your laptop to checks on every production run

1. Unit
Transform logic on tiny in-memory data

2. Integration
Real formats & stores on data samples

3. Quality gates
Expectations validate every real run

4. Observe
Freshness, volume & lineage monitored

tests/test_clean_events.py — CI on every push

```
def test_dedup_keeps_latest():
  df  = spark.createDataFrame(SAMPLE)
  out = clean_events(df)
  assert out.count() == 3
  assert no_nulls(out, "order_id")
# late & duplicate events in SAMPLE
# CI: proves the code meets these rules on sample data
# prod: the same rules re-checked on real data every run
```

Failure modes to avoid

- **Happy-path testing** — pipelines die on nulls, late data & schema drift; test exactly those
- **Fixtures that rot** — refresh samples from real production shapes regularly
- **Green CI, red production** — unit tests never replace run-time quality gates
- **Untested recovery** — rehearse a backfill on staging before you need one at 3 a.m.

NEW TERMS

`CI` Continuous Integration: tests run on every push
`Integration test` test against real formats and stores
`Happy path` the case where nothing goes wrong
`Schema drift` source columns change without notice

AI-Assisted Quality Rules & Docs

✓ Where assistants shine

- **Profiling** — drafts an expectation suite from a data sample in seconds
- **Coverage** — proposes edge cases you forgot: nulls, ranges, drift
- **Docs** — turns DAGs & suites into readable pipeline documentation
- **Consistency** — one rule style across every dataset in the platform
- **Speed** — a first quality suite in minutes, not afternoons

	Hand-written complete control, slow	Blind AI rules generated, unreviewed	AI + your audit draft · verify · own
COVERAGE	✗ Sparse	✓ Broad	✓ Broad
TRUST	✓ You wrote them	✗ Plausible ≠ true	✓ Validated on data
EFFORT	✗ Hours / table	✓ Minutes	✓ Minutes + review

Assistants draft the rules and the docs — you validate both against real data before they gate production.

NEW TERMS

- `Profiling` summarising a dataset's values
- `Expectation suite` all the rules for one dataset
- `Drift` data distribution shifts over time

The Orchestration Ecosystem

The orchestrators & quality engines

Apache Airflow
the industry-standard DAG platform

Dagster
asset-aware, data-first orchestration

Prefect
lightweight, Python-native flows

dbt
SQL transformations with built-in tests

Great Expectations
declarative data-quality suites

Where each one fits

Define
Declare tasks & dependencies as code — in git, reviewed like software

Airflow · Dagster · Prefect · dbt

Schedule
Time- or data-triggered runs, each bound to its logical date

Airflow · Dagster · Prefect

Execute & recover
Retries, pools and backfills keep every run healthy at scale

Airflow pools, retries & backfills

Validate & observe
Quality gates, lineage and alerts guard every promotion

Great Expectations · dbt tests · Soda · plain SQL · Python code

`In this week's lab:` you will wire an Airflow 3 DAG with quality gates over a multi-stage pipeline — all inside the course compose file.

NEW TERMS

- `Dagster` asset-centric orchestrator
- `Prefect` Python-native workflow orchestrator
- `dbt` SQL transformation tool with tests
- `Expectation` declarative rule data must satisfy (any tool)

Reading List

Ranked by priority — the primary set is what the lab assumes you have read; the secondary set goes deeper on the slides listed.

1 · Primary reading — read before the lab

1 **Reis & Housley, *Fundamentals of Data Engineering*, 2022**
Ch. 2 undercurrents, ch. 8 ETL vs ELT · Slides 6–11

2 **Apache Airflow docs, *Core Concepts*, DAGs to Sensors**
DAGs, tasks, operators, sensors, schedules · Slides 12–21

3 **Beauchemin, *Functional Data Engineering*, 2018**
Idempotent tasks, partition overwrite, backfills · Slides 22–26

4 **Great Expectations docs, *Expectations & Checkpoints*, GX Core**
Validation gates between pipeline stages · Slides 27–31

2 · Secondary reading — go deeper

1 **Kleppmann, *Designing Data-Intensive Applications*, ch. 10**
Batch workflows and idempotent outputs · Slides 9–10, 24

2 **Harenslak & de Ruiter, *Data Pipelines with Apache Airflow*, 2021**
Scheduling, incremental loads, backfilling · Slides 14–26

3 **Armbrust et al., *Delta Lake*, VLDB 2020**
ACID tables, atomic overwrite on object stores · Slides 8, 11, 24

4 **Databricks docs, *Medallion Architecture*, 2024**
Bronze, silver and gold layers · Slide 11

5 **Schelter et al., *Automating Data Quality Verification*, VLDB 2018**
Declarative checks at scale (Deequ) · Slides 27–29, 31

6 **dbt docs, *Data Tests*, dbt Core**
Tests in CI for SQL transformations · Slides 30, 32

Read the primary set in order; pick secondary items as your lab job needs.

PART 2 · HANDS-ON

Lab

06